

Technical Review & Build Verdict

Financial Forecasting by Multi-AI Agent System — B.Tech Capstone

Reviewer role: Finance-AI research review

Review date: 23 July 2026

Scope: [BTP_All_Phases_Work_Breakdown.pdf](#) (5 phases), existing repository state, and the underlying research premise

Method: Every factual claim below was checked against a primary source or reproduced on this machine. Claims that could **not** be verified are marked **[!] UNVERIFIED** and listed separately in §9 — they are not presented as findings.

1. Verdict

[OK] BUILD — but do not build the plan as written.

The architecture is sound and the engineering instincts are good. The project does **not** need a restart. However, **as currently planned, the project would pass its demo and fail its evaluation chapter**. Three things must be fixed *before* Phase 2 code is written, because retrofitting them later means touching every tool and every agent:

#	Must fix before building	Why it cannot wait
1	Add <code>as_of</code> to every data function	Retrofitting means rewriting all 6 tools <i>and</i> re-running all experiments
2	Move evaluation before the build; define a post-cutoff test window	A within-training-window backtest is invalid; discovering this in Phase 5 leaves no time
3	Reposition the novelty claim against TradingAgents	Affects the report's framing, the research question, and what you measure

Everything else in this review is fix-as-you-go.

Estimated cost of these three fixes: ~1 day of planning. Cost of not fixing them: the results chapter.

2. What Was Verified, and How

#	Claim under test	Method	Result
1	<code>pandas-ta-classic</code> import name	PyPI package page	[FAIL] Bug confirmed — imports as <code>pandas_ta_classic</code>
2	Project venv state	Direct <code>site-packages</code> inspection	[FAIL] 0 packages installed
3	<code>tests/test_env.py</code> runs	Executed on this machine	[FAIL] All 5 checks fail
4	yfinance rate ceiling " <code>~2,000/hr</code> "	Maintainer issue tracker	[FAIL] False — <code>~360/hr</code> ; 429s reported from <code>~950</code>
5	LangGraph parallel state writes	LangChain error docs	[OK] Confirmed — needs <code>Annotated</code> reducer

#	Claim under test	Method	Result
6	Gemini nested-Pydantic reliability	google-genai + pydantic-ai issue trackers	[!] New risk found (see §4.4)
7	Gemini free-tier RPM/RPD	Official rate-limit docs	[!] Not publishable — Google no longer lists numbers
8	Lookahead bias is real & measurable	arXiv:2512.23847	[OK] Confirmed
9	Architecture already published	arXiv:2412.20138 + GitHub	[OK] Confirmed
10	NewsAPI free-tier viability	Vendor pricing pages	[OK] Confirmed unusable for backtest
11	GDELT global + free	gdeltproject.org	[OK] Confirmed — 100+ languages, 1979 -> present
12	yfinance global coverage	Yahoo exchange list	[OK] Confirmed — global via suffixes
13	FRED needs its own key	FRED API docs	[OK] Confirmed — separate registration
14	Recharts candlestick support	Library docs	[OK] Confirmed — no native candlestick

3. [CRITICAL] Blocking Bugs — Reproduced on This Machine

These are not opinions. They were executed.

3.1 The environment is empty

```
venv/Lib/site-packages -> 0 non-pip packages
```

The virtualenv exists; `pip install` never completed. **Phase 1 Task 1.2 is not done**, despite the folder structure suggesting otherwise.

3.2 Every import fails

```
FAIL yfinance    FAIL pandas_ta    FAIL google-genai    FAIL langgraph
FAIL config error: No module named 'backend'
```

3.3 `import pandas_ta` will fail even after installing correctly **[!]** Highest-value catch

`tests/test_env.py:14` reads `import pandas_ta as ta`.

But `pandas-ta-classic v0.6.52` imports as `pandas_ta_classic` — verified on PyPI:

```
import pandas_ta_classic as ta    # OK correct
import pandas_ta as ta           # FAIL ModuleNotFoundError
```

Without this fix, the test reports failure forever and you conclude the install is broken when it isn't. Also note: **requires Python >= 3.10.**

Fix:

```
import pandas_ta_classic as ta
```

3.4 No module named 'backend'

Two causes: `backend/` has no `__init__.py`, and running `python tests/test_env.py` puts `tests/` on `sys.path`, not the project root.

Fix: add `backend/__init__.py` and run as a module from the project root:

```
python -m tests.test_env
```

4. [CRITICAL] Design & Correctness Problems

4.1 The data layer leaks the future by construction

Phase 2 specifies:

```
get_price_history(ticker, period)
get_fundamentals(ticker)
get_news(ticker)
get_macro_data()
```

None accept a date. They always return *now*. Phase 5 then asks you to "pretend today is 3 months in the past."

The result: your sentiment agent reads **this week's** headlines — which already contain the outcome — while claiming to forecast from three months ago. That is not forecasting; it is hindsight. A panel finds this in under a minute.

Fix — build into Phase 2, not Phase 5:

```
get_price_history(ticker, start, end, as_of)
get_fundamentals(ticker, as_of)           # filter to filings published <= as_of
get_news(ticker, as_of)                  # filter to published_date <= as_of
get_macro_data(as_of)                   # FRED series respecting vintage
```

Enforce the cutoff **in the data layer**, never by prompting the model.

4.2 The backtest is invalid even after 4.1 is fixed [CRITICAL] Most important finding

This is the point most reviewers stop one step short of.

Fixing `as_of` removes leakage from the **pipeline**. It does nothing about leakage inside the **model's weights**. Gemini has already read what happened to these stocks. Per [arXiv:2512.23847](#), *Lookahead Propensity* is materially positive throughout the training window and **collapses to ~0 immediately after the training cutoff**.

So a point-in-time-correct backtest over a within-training-window period is still meaningless.

Fix (both are required, not either/or):

- 1 `as_of` plumbing everywhere (§4.1), and
- 2 Test window placed **strictly after the model's training cutoff**, confirmed with a recall probe: ask the model about outcomes it should not know and verify it cannot recall them. Publish that probe as a figure — it is a genuine methodological contribution.

4.3 Cache design defeats its own purpose

The plan says: *cache is fresh if < 15 min old*.

For historical evaluation data keyed by an `as-of` date, **the value never changes**. A 15-minute TTL forces constant re-fetching during ablation sweeps — causing exactly the 429s the cache exists to prevent.

Fix — two cache classes:

Class	Key	TTL
Historical / point-in-time	(ticker, field, as_of)	Permanent

Class	Key	TTL
Live quote (demo only)	(ticker, field)	15 min

Cache **LLM responses** too, keyed by a prompt hash. Debugging re-runs must not consume fresh quota.

4.4 [!] Gemini structured output will fight your nested schemas — new risk, not previously flagged

Task 3.1 defines `FinalForecast` with nested sub-objects. Verified issues in Google's own tracker:

- **Gemini rejects Pydantic schemas that use `Field(default=...)`** (googleapis/python-genai #699). Your schemas will almost certainly use defaults.
- **Nested Pydantic models can be misread as tool calls**, producing `Unknown tool name: 'NestedModel'` or `MALFORMED_FUNCTION_CALL`.
- `propertyOrdering` improves output quality by up to ~15% on complex schemas — and models do *not* respect your declared field order without it.

Fix:

- 1 Test `FinalForecast` — your most complex schema — against the API in **Phase 1**, not at integration time.
- 2 Avoid `Field(default=...)` in response schemas; use `Optional[...]` and post-process.
- 3 Set `propertyOrdering` explicitly.
- 4 Keep nesting shallow — prefer flat schemas with prefixed field names over deep object trees.
- 5 Count parse failures as a **reported metric**, not a silent retry.

4.5 Parallel agents will crash LangGraph without a reducer

Task 3.5 fans out 4 specialists in parallel. If two nodes write the same state key in one step, LangGraph raises `INVALID_CONCURRENT_GRAPH_UPDATE` — *"Can receive only one value per step."*

Fix — either:

```
# (a) distinct keys per agent ← simplest, recommended
class AgentState(TypedDict):
    technical_report: TechnicalReport | None
    fundamental_report: FundamentalReport | None
    sentiment_report: SentimentReport | None
    macro_report: MacroReport | None

# (b) or a reducer, if they share a key
from typing import Annotated
import operator
reports: Annotated[list, operator.add]
```

4.6 Smaller confirmed defects

Issue	Impact	Fix
SMA-200 on a 1M window	Silent NaN	Always fetch ≥ 2 years regardless of chart range
Stooq $\neq$ yfinance schema	Fallback breaks during the outage it exists for	Normalise columns/adjustments at the boundary; unit-test the fallback path
yfinance nulls on non-US tickers	Agent crashes or invents a number	Explicit null handling; agents must say "unavailable", never guess
No news dedup	Same story double-counts, skewing polarity	Dedup by URL + title similarity

Issue	Impact	Fix
No version pinning	LangGraph breaking changes	<code>pip freeze > requirements.txt</code> after first good install
FRED key not provisioned	Blocks Task 2.6 mid-Phase 2	Register in Phase 1 alongside Gemini
Recharts has no candlestick	Fighting the library	Use TradingView Lightweight Charts
POST vs WebSocket unowned	30–90 s request times out on proxies	WebSocket owns the live flow; POST only for cached results
Non-deterministic runs	Reported numbers shift before demo day	temperature~0 on synthesis; cache the actual scored run

5. The Research Problem

5.1 The architecture is already published

TradingAgents: Multi-Agents LLM Financial Trading Framework — arXiv:2412.20138

Technical / fundamental / sentiment analysts, Bull and Bear researchers that debate, a risk management team, a synthesising trader. Public repo:
github.com/TauricResearch/TradingAgents.

That is this project's design. **"Our contribution is the Bull vs. Bear debate" will not survive a literature review.**

This is *not* fatal. Replication-plus-extension is legitimate and often stronger than a weak novelty claim. But it must be cited and positioned deliberately, in Chapter 2.

5.2 "Proving multi-agent > single LLM" is the wrong goal

The Phase 5 deliverable reads: *"Backtest results **proving** multi-agent > single LLM."*

You cannot set out to prove a conclusion. If the debate layer doesn't help, that is a **result**, not a failure — and a well-measured negative result defends better than a vague positive one.

5.3 Where the real contribution lives

Axis	Why it is defensible
Leakage-free evaluation	Most published multi-agent finance results do not control for it
Debate ablation	Does the debate earn its ~2x token cost? Almost nobody measures this
Confidence calibration	When it says 80%, is it right 80% of the time? Rarely reported
Global coverage	The literature is overwhelmingly US-large-cap (see §6)

Proposed research question:

Under an evaluation protocol that provably excludes lookahead bias, does adversarial multi-agent debate improve forecast accuracy and confidence calibration over (a) a single-LLM baseline and (b) the same specialists without debate — and is any gain justified by its token cost?

Answerable either way. That is what makes it a research project.

6. Worldwide Free Data Strategy

Requirement noted: global coverage, free, not implemented yet. This section is design-only.

Good news: fully global coverage is achievable at zero cost.

6.1 Price data — yfinance covers the world via exchange suffixes

Market	Suffix	Example
USA	(none)	AAPL
India NSE	.NS	RELIANCE.NS
London	.L	BP.L
Tokyo	.T	7203.T
Germany XETRA	.DE	SAP.DE
Hong Kong	.HK	0700.HK
Australia ASX	.AX	BHP.AX
Paris	.PA	AIR.PA
Toronto	.TO	RY.TO

Yahoo maintains the authoritative suffix list. **Design the ticker layer to accept any suffix from day one** — hardcoding a market is the mistake to avoid.

6.2 News — GDELT is the correct global choice

Verified: **100+ languages, every country, archive from 1 Jan 1979, updates every 15 minutes, 100% free**, queryable via BigQuery.

This solves what NewsAPI cannot: NewsAPI's free tier is **100 req/day, 1-month archive, 24-hour delay, localhost-only** — unusable for any historical evaluation.

Trade-off to document: GDELT gives tone/theme/metadata, **not full article text**. Adequate for a sentiment agent; state it as a limitation.

6.3 Recommended stack

Layer	Source	Cost	Global?
Price / OHLCV	yfinance (cached) -> Stooq fallback	Free	[OK]
Bulk history	Exchange EOD archives	Free	[OK]
Indicators	Computed locally	Free	[OK]
Fundamentals	yfinance [!] <i>not point-in-time</i>	Free	Partial
News	GDELT	Free	[OK]
Macro	FRED (+ regional central banks)	Free	[OK]

Total data cost: zero.

6.4 The one gap money cannot close

Point-in-time fundamentals. yfinance returns *today's* restated financials, not what was public on your as-of date. True PIT databases (Compustat PIT, FactSet) are institution-priced.

Mitigation, not purchase: lag fundamentals ≥ 1 quarter after the reporting date, restrict to slow-moving ratios, and **document the limitation explicitly**. A stated limitation earns credit; a hidden one loses it.

7. Security Review

Relevant because this is a project *about* AI agents — examiners will ask.

Risk	Severity	Fix
Prompt injection via headlines — untrusted web text goes straight into the LLM	Medium	Treat headlines as data, not instructions: delimit clearly, instruct the model to ignore embedded directives, validate output against the schema. Discuss in the report — it shows awareness of the risk class.
No auth / rate limit on FastAPI	High <i>if deployed</i>	Per-IP rate limit before any public exposure; otherwise anyone can drain your quota
<code>NEXT_PUBLIC_</code> key leak	Critical	Never prefix secrets with <code>NEXT_PUBLIC_</code> — it ships them into the client bundle. All provider calls stay server-side.
<code>**CORS allow_origins=["*"]**</code>	Medium	Pin to your frontend origin explicitly
WebSocket has no timeout/connection cap	Low	Idle timeout + max connections
<code>.env</code> committed	Critical	<code>.gitignore</code> exists — verify the key was never committed before pushing
No dependency audit	Low	<code>pip audit</code> before submission

8. [OK] Corrected Phase Plan

The count is not the issue — **the order is**. Build the ruler before the thing it measures.

Phase 1 — Foundation & Walking Skeleton

Environment (fixing §3), **all** API keys including **FRED**, `cache.py` first, `market_data.py` with `as_of` + Stooq fallback, indicators, **the single-LLM baseline**, one specialist end-to-end, **and a FinalForecast schema smoke-test against Gemini** (§4.4).

Gate: `python -m forecast <TICKER> --as-of <DATE>` **returns a valid schema-conformant forecast twice; the second run makes zero network calls.** `pip freeze > requirements.txt` committed.

Phase 2 — Evaluation Harness & Leakage-Free Protocol

Determine the training cutoff, run the recall probe, **freeze and commit the test set**, implement metrics (directional accuracy, MAE, **calibration/Brier**, **cost per forecast**), add a naive baseline, record baseline results.

Gate: any forecaster can be scored by one command, reproducibly; probe result documented.

Phase 3 — Multi-Agent System

Remaining specialists, LangGraph StateGraph with **distinct state keys** (§4.5) and checkpointing, **reconciliation gate**, bounded Bull/Bear debate, risk synthesiser, and the `--no-debate` **switch wired from the start**.

Gate: full system scores end-to-end; reconciliation catches a deliberately injected fake number.

Phase 4 — Experiments & Ablation ← *where the marks are*

Multi-agent vs single-LLM vs naive; **debate ablation** (accuracy delta **and** cost delta); specialist leave-one-out; calibration curves; ≥ 3 repeats for variance; qualitative failure analysis on 5–10 wrong forecasts.

Gate: every claim destined for the report traces to a committed results file.

Phase 5 — Dashboard, Report & Defence

FastAPI + **Streamlit first** (React only if time genuinely remains), live agent log, debate view, forecast card with citations. Report with **TradingAgents positioned in Chapter 2**. Slides + **a cached known-good demo run**.

Gate: clean clone runs the demo; every number traces to a results file.

9. [!] What I Could NOT Verify

Listed explicitly so the next reviewer can close them.

Item	Status	Action
Gemini free-tier RPM/RPD	Google's official rate-limit page no longer publishes per-model numbers ; it directs users to AI Studio	Check aistudio.google.com directly before designing around any figure. Treat all quoted numbers — mine or anyone's — as unverified.
Gemini training cutoff date	Not reliably published per model	Determine empirically via the §4.2 recall probe. This is more defensible than citing a date anyway.
Exact yfinance ban threshold	Undocumented by design (unofficial scraper); community reports vary	Assume it is <i>lower</i> than any number you read. Cache-first makes it moot.
TradingAgents' exact datasets/results	Abstract lacks detail	Read the full PDF before writing Chapter 2

Design consequence: because the two most load-bearing numbers (LLM quota, training cutoff) are not reliably published, the architecture must be **quota-adaptive** (backoff + caching + Flash-class models) and the evaluation must be **empirically probed**, not documentation-driven.

10. Bottom Line

Should you start building? Yes — today.

- The architecture is validated by both industry practice and published research.
- The engineering plan is largely sound; most defects here are fix-as-you-go.
- Nothing requires a restart.

But do these three things first, in this order:

- 1 **Fix the environment** (§3) — ~10 minutes. `import pandas_ta_classic, backend/__init__.py, pip install, pip freeze.`
- 2 **Add `as_of` to every tool signature** (§4.1) — before writing any Phase 2 body. Cheap now, expensive later.
- 3 **Fix the evaluation protocol and novelty framing** (§4.2, §5) — ~1 day. This is what stands between a working demo and a defensible thesis.

The honest summary: the previous plan builds a system that *works* but cannot be *evaluated*. The demo would impress; Chapter 5 would not survive questioning. The corrected plan produces both — and the extra cost is roughly one day of planning, paid now instead of a lost results chapter paid later.

References

- TradingAgents: Multi-Agents LLM Financial Trading Framework — [arXiv:2412.20138](#)
- Detecting Lookahead Bias in LLM Forecasts — [arXiv:2512.23847](#)
- Summoning the Oracle to Slay It: Mitigating Look-Ahead Bias in Financial Backtesting with LLMs — [arXiv:2605.24564](#)
- LangGraph — `INVALID_CONCURRENT_GRAPH_UPDATE` error reference, LangChain docs
- Gemini structured output — [googleapis/python-genai issue #699](#); [pydantic-ai issue #3483](#)
- pandas-ta-classic — PyPI, v0.6.52
- GDELT Project — [gdeltproject.org](#)
- Yahoo Finance exchange & suffix list — [help.yahoo.com](#)

*Prepared for independent expert review. Findings marked **[OK]** were verified against primary sources or reproduced locally; items in §9 are explicitly open.*